

P3087R1

Make direct-initialization for enumeration types at least as permissive as direct-list-initialization

Published Proposal, 2024-05-29

This version:

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3087r0.html>

Author:

[Jan Schultke](#)

Audience:

EWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Source:

[eisenwave/cpp-proposals](#)

Abstract

This paper proposes allowing initializing scoped enumeration types using direct-initialization in those cases where direct-list-initialization is already possible.

Table of Contents

1 Revision history

1.1 Changes since R0

2 Introduction

3 Motivation and scope

4 Impact on existing code

5 Design considerations

- 5.1 Fixed underlying types
- 5.2 Floating-point initializers
- 5.3 Implicit conversions
- 5.4 Considered alternatives
- 5.5 Radical permissiveness

6 Implementation experience

7 Proposed wording

References

Normative References

Informative References

§ 1. Revision history

§ 1.1. Changes since R0

- Discuss alternative designs.
- Strengthen rationale for proposed design.

§ 2. Introduction

Currently, the following code is ill-formed:

```
enum class E {};  
E a{0}; // OK, list-initialization  
E b(0); // error: cannot convert 'int' to 'e' in initialization
```

There is no obvious reason why direct-list-initialization must be more permissive than non-list-direct-initialization in this case. I propose to make this code well-formed.

§ 3. Motivation and scope

During the discussion leading up to this proposal, multiple developers were bewildered by the status quo. Direct-list-initialization is perceived as a stricter form of direct-initialization; in essence *"direct-initialization without narrowing conversions"*. Making direct-initialization at least as permissive as direct-list-initialization would validate that intuition.

It's confusing that `static_cast<std::byte>(0)` and `std::byte(0)` are permitted, but `std::byte b(0)` and `new std::byte(0)` are not. It is difficult to come up with a simple and intuitive rule that would explain why this is the case; at least it would require pointing out the equivalence of functional notation explicit conversions and "C-style casts" in the specific case where the initializer is a parenthesized single expressions. I see this as far from teachable.

[P0960R3] proposed to expand parenthesized initialization capabilities for aggregate types. That proposal was motivated by perfect forwarding of aggregate types. Similarly, this proposal would improve ergonomics in cases such as:

```
std::vector<std::byte> bytes;  
bytes.emplace_back(0xff);           // currently ill-formed, proposed to be OK
```

[P0138R2] proposed direct-list-initialization for enumeration types. It did not discuss whether non-list-direct-initialization of enumeration types should be made more permissive in tandem.

§ 4. Impact on existing code

This code only makes previously ill-formed initialization valid. Naturally, this impacts any uses of expression testing (SFINAE, requires-expression, etc.).

`std::is_constructible_v</* enumeration type */, /* ... */>` and other traits that test for validity of `T t(expr)` are impacted.

§ 5. Design considerations

This proposal largely adopts the rules for direct-list-initialization. It merely drops the requirement of non-narrowing conversions.

§ 5.1. Fixed underlying types

As with direct-list-initialization, a fixed underlying type should be required. Enumerations without fixed underlying types act as symbolic constants in the program, or are used as a C-compatible non-macro way to define constants. There is no motivation to expand direct-initialization rules for these types.

NOTE: All scoped enumerations implicitly have an `int` (or wider integer) fixed underlying type.

§ 5.2. Floating-point initializers

The proposal seeks to make the following code valid:

```
std::byte b(0.f);
```

This construct is undesirable, but necessary to achieve the *"at least as permissive as direct-list-initialization"* semantics.

§ 5.3. Implicit conversions

This proposal does not seek to make implicit conversions from scalar to enumeration types possible. The following code is and should remain ill-formed:

```
void foo(std::byte);  
foo(0);           // ill-formed  
std::byte b = 0; // ill-formed
```

Allowing implicit conversion to enumerations in general would compromise the type safety that enumerations offer. Direct-initialization is a very specific case where intent is relatively clear.

§ 5.4. Considered alternatives

Besides the proposed mechanics, there are a few alternatives, which are compared below:

1. Leave everything as is (status quo).

2. Make parenthesized initialization behave exactly as list initialization.
3. Restrict specific conversions (e.g. disallow floating-point to enumeration).

Option	<code>std::byte b(0);</code>	<code>std::byte b(-1);</code>	<code>std::byte b(0.f);</code>
Status quo	✗	✗	✗
Like list-init	✓	✗	✗
Restrict some	✓	✓	✗
Proposed	✓	✓	✓

§ 5.5. Radical permissiveness

The proposed design is radically permissive, especially noting [§ 5.4 Considered alternatives](#). This comes with a certain potential for writing bugs.

However, any restriction beyond the current design would somewhat defeat the purpose of this proposal. It would be novel design that is inconsistent with how non-list-direct-initialization usually works, which is the problem we're trying to solve.

Furthermore, these pitfalls could be seen as quality-of-implementation issues, not as language issues. It is possible to emit compiler warnings when enum initialization is performed that would be considered narrowing.

Last but not least, the permissiveness can be justified by considering that direct-initialization is a rare style that implies an intent to opt into such mechanics. For example, if scoped enumerations did not exist and were emulated using classes, this would likely imply having a constructor of the form `explicit Enum(underlying)`, which could also only be used with direct-initialization. Therefore, the current proposal makes enumerations more symmetrical with class types.

§ 6. Implementation experience

None.

§ 7. Proposed wording

The proposed changes are relative to the working draft of the standard as of [\[N4917\]](#).

Insert a new bullet in 9.4.1 [dcl.init.general] paragraph 16, between bullets 7 and 8:

Otherwise, if

- the destination type T is an enumeration with a fixed underlying type ([dcl.enum]) U ,
- the parenthesized *expression-list* of the *initializer* has a single element v of scalar type, and
- v can be implicitly converted to U ,

the object is initialized with the value `static_cast<T>(v)` ([expr.type.conv]).

NOTE: This bullet covers all forms of direct-initialization except list-initialization and `static_cast`. List-initialization is already covered by an earlier bullet, and `static_cast` has no *expression-list*, only a single *expression*.

Modify 9.4.5 [dcl.init.list] paragraph 3 bullet 8 as follows:

~~Otherwise, if T is an enumeration with a fixed underlying type ([dcl.enum]) U , the *initializer-list* has a single element v of scalar type, v can be implicitly converted to U , and the initialization is direct-list-initialization,~~

Otherwise, if

- T is an enumeration with a fixed underlying type ([dcl.enum]) U ,
- the *initializer-list* has a single element v of scalar type,
- v can be implicitly converted to U , and
- the initialization is direct-list-initialization,

the object is initialized with the value ~~`T(v)` ([expr.type.conv])~~ `static_cast<T>(v)` ([expr.static.cast]); if a narrowing conversion is required to convert v to U , the program is ill-formed.

NOTE: This change is strictly editorial. It would be valid to keep list-initialization defined in terms of `T(v)` instead of `static_cast<T>(v)`. However, the semantics of `T(v)` in [expr.type.-conv] are delegated to `static_cast` in [expr.static.cast] anyway, which is an unnecessary double indirection.

§ References

§ Normative References

[N4917]

Thomas Köppe. *Working Draft, Standard for Programming Language C++*. 5 September 2022.

URL: <https://wg21.link/n4917>

§ Informative References

[P0138R2]

Gabriel Dos Reis; Microsoft. *Construction Rules for enum class Values*. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0138r2.pdf>

[P0960R3]

Ville Voutilainen; Thomas Köppe. *Allow initializing aggregates from a parenthesized list of values*. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0960r3.html>